

Technical Report

1. Introduction & business context

Today, e-commerce businesses rely heavily on using data-driven analytics to improve their business practices, decrease costs through better understanding of their customer's needs, and increase revenues. One of the companies that will be discussed in this case study is based in the UK and sells an extensive range of unique gifts for all occasions to both wholesalers and consumers. Between December 2010 and December 2011, the company compiled about 500,000 transactions of all types within its database, this data provides a wealth of information regarding sales, products, as well as the total number of customers located in each geographic area based on the purchasing habits reflected in the transaction data.

Large-scale internet retailers like Amazon experience challenges that present themselves through:

- Cancellations, returns and fraudulent transactions
- Detecting unusual behavior in terms of customer purchasing behavior and high-value customer identification
- Identifying extreme price points and data voids by anomaly detection
- Ensuring the quality of the data generated from analytics and reports

This project will provide an opportunity to simulate an actual real-world e-commerce transactional big data pipeline that provides analysis-ready data from raw e-commerce transactional data. This will be accomplished using PySpark on Google Colab as the large-scale processing platform and MongoDB Atlas as the NoSQL database to store the curated high-performance dataset. More specifically, the pipeline be able to do the following:

- Store raw transaction records in a database (can be referred to as “bronze layer”) without any transformation or cleaning of the path into a traditional analytical data mart database.
- Clean and validate data using strong data cleansing and validation techniques (so-called “silver layer” of the ETL data pipeline) for missing Customer IDs, negative orders, cancelled orders, negative order total, and duplicate records.
- Create analytical features such as revenue measurements, multi-item order features (baskets), time initiated attributes, and customer segments using RFM to form the “golden layer” of an ETL data pipeline.
- Save created data sets in a MongoDB document database that is optimized for retrieval and aggregation so that they can be used for meaningful business analytics.

The completed pipeline will yield a scalable, reproducible, and performance efficient analytic system to assist the business in discovering critical questions;

- Who spends the most money with my business?
- What are the most popular items purchased by my customers year round?
- How do returns and cancellations affect the business' total net sales?

- What geographical areas are generating the most sales?

The completion of the pipeline will demonstrate industry standard ETL practices, distributed analytics approaches, NoSQL data modeling, and performance enhancements, all of which are vital elements for data engineers and analytic teams.

2. Dataset description

2.1 Dataset Source and Overview

The data used for this project were collected from Kaggle and includes records of transactions made by a UK online retail business for all transactions made from 01 December 2010 until 09 December 2011.

Products sold by this company include unique all occasion gift items to both private customers and wholesalers. Over half a million transactional records were included in the dataset (real life e-commerce transactions including purchases, cancellations and returns).

Due to the size and transactional nature of the dataset, the dataset is ideally suited for creating an Apache Spark-based distributed big data processing pipeline and storing the resulting curated-based output in MongoDB.

2.2 Dataset Structure and Attributes

The dataset is made up of 8 main features which describe transactional activity at the product-line level. Each row of the dataset corresponds to an individual product (line item) included in each invoice - therefore there may be multiple rows per invoice.

The following lists describe each feature in more detail.

- **InvoiceNo** - A 6-digit nominal identifier that uniquely identifies all transactions. Invoice numbers that begin with the letter 'C' signify that the transaction has been cancelled.
- **StockCode** - A 5-digit nominal identifier which uniquely identifies each different product.
- **Description** - Textual attribute which holds the product's description.
- **Quantity** - Numeric attribute which indicates how many units were purchased during a transaction. There will be negative values for products that are returned.
- **InvoiceDate** - A timestamp attribute which indicates when the transaction occurred.
- **UnitPrice** - Numeric attribute which indicates the price of each unit in GBP.
- **CustomerID** - A 5-digit nominal identifier which uniquely identifies each customer. There are some NULL values in this attribute.
- **Country** - Nominal attribute that identifies which country the customer lives in.

2.3 Characteristics of the Dataset

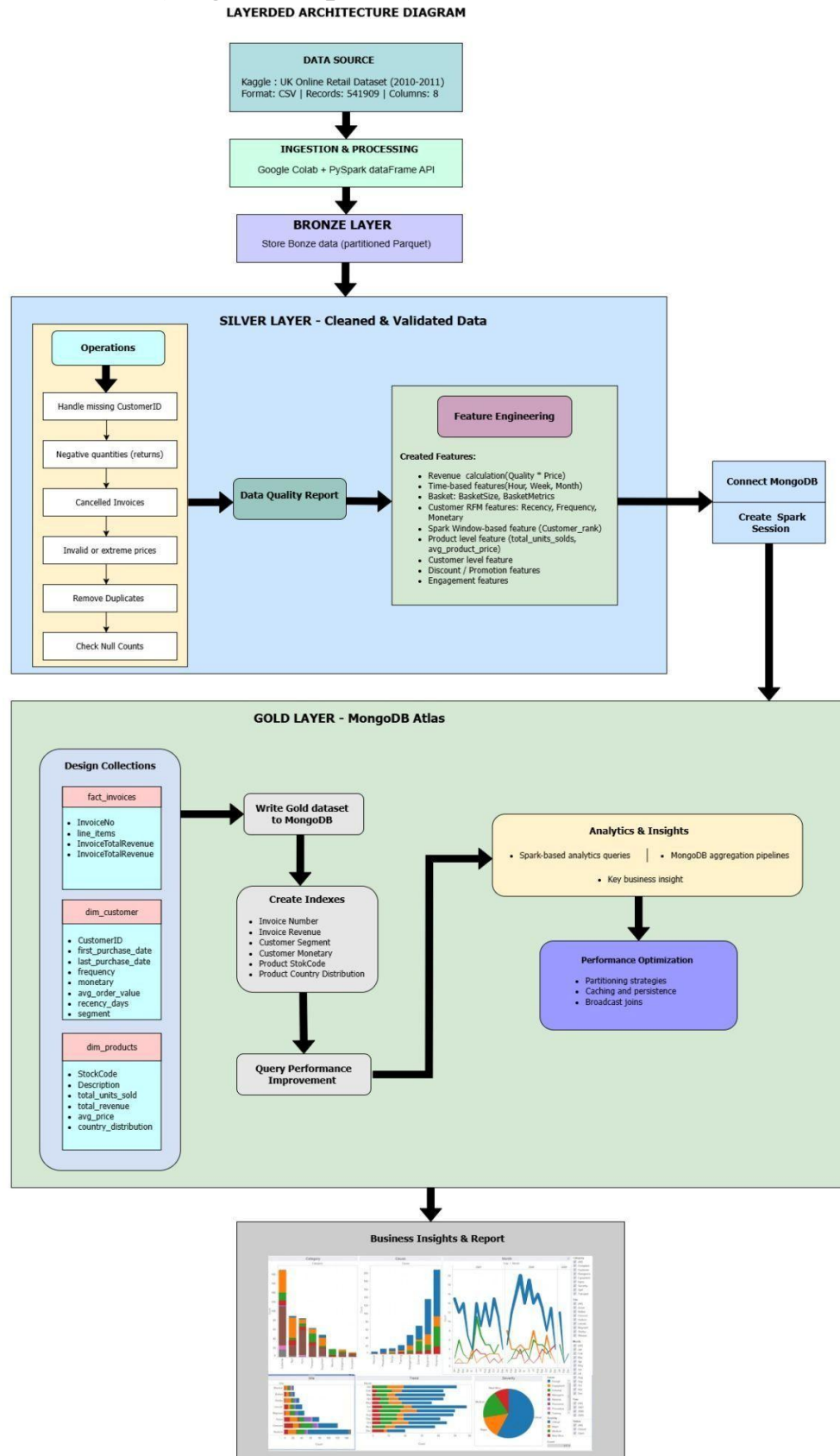
The following counts reflect the dataset characteristics:

- 25,900 unique invoice numbers
- 4,070 unique product codes
- 4,224 unique descriptions for products sold
- 4,373 unique customers
- 38 countries
- 23,260 unique invoice timestamps

Based upon these statistics, we can conclude that there is a wide diversity of products and an international customer base. In addition to having multiple products for each invoice, having a one-to-many invoice to item relationship makes this type of information ideal for document-oriented data modeling in a NoSQL database.

The dataset is transactional, with invoices containing multiple product items. The dataset also contains cancelled invoices, records of product returns with negative quantities, many records without a Customer ID, duplicate records, and extreme value records. All of these data quality issues can affect both revenue calculations and customer analysis. Additionally, the dataset contains more than 500,000 records with a multi-attribute structure which makes it very suitable for distributed processing. The dataset supports key analytical functions including revenue computations, basket analysis, customer segmentation, and geographical sales aggregation. Overall, the dataset is an excellent basis for building a scalable, multi-layered e-commerce data pipeline.

3. System architecture (diagram required)



The system has been designed using a layered ETL (Extract, Transform, Load) architecture following the industry standards for large quantities of transactional data from e-commerce websites. The architectural structure has three layers (Bronze, Silver and Gold). Each layer provides a progressively refined level of data from raw ingestion (Bronze) to analytical-ready data sets (Gold).

Bronze Layer - Raw Data

The raw data layer (Bronze Tier) is where the original data is still in place with the least amount of transformation applied to it. The purpose of this layer is to keep the transactional (raw) data as it was received and to have a record of history and be able to reprocess the data in the future.

The dataset is being loaded into PySpark via the export function with a file type of Parquet. The Parquet file is partitioned by year and month in accordance with the invoice date. The benefit of partitioning the data will increase query performance and allow for the support of a scalable storage solution as data grows.

As the Source of Truth, the Bronze data layer maintains a complete and safe record of all the raw transactional data that has been ingested. No data cleansing, filtering or validation is completed in the Bronze layer, and all the data will be used in its original state for the downstream processing in the Silver and Gold data layers.

Silver Layer - Cleaned & Validated Data

The Silver Layer is where the cleaning and validation of data in the pipeline take place. Data within this layer is transformed from its raw state in the Bronze layer into a format suitable for analysis that is clean, reliable, and adequate for analytical purposes. Another goal of this layer is to improve the quality of the data and preserve useful transactional information.

In contrast to the Bronze layer, at this stage there are structured cleaning rules applied to the data, validation rules applied to the data, and several data quality measurement tools are utilized to ensure that any inaccurate, incomplete or inconsistent records do not create harm to the analyses being processed in the downstream analytics.

Gold Layer - Analytics-Ready Data

Cleaning up the data is done during the Gold phase, with the cleaned data being structured for analytics. This project uses MongoDB Atlas rather than a traditional relational warehouse to store collections that are ready for analytics in a NoSQL format.

The following three collections were created,

Fact_invoice - These collections hold an invoice's line-item data. By embedding line items inside each invoice record, this eliminates the need for joins and helps the aggregation process perform better.

Dim_customer - These collections will have the recency, frequency, and monetary value (RFM) of every customer for segmentation and marketing analysis.

Dim_product - These collections will contain the performance data of products such as total sales and quantity sold.

Queries can be executed faster because of the indexes that were created on fields such as the customer id, invoice date, stock code, and segment.

To begin with, the purpose of the Gold phase is to provide the best-structured and most-optimized way to get curated data into analytical/sub-analytical uses.

This structured model uses a Bronze, Silver, and Gold (Bronze/Silver/Gold) system architecture to provide progressive data refinement from initial raw ingestion through to finished analytics-ready data models. The integration between PySpark and MongoDB creates a layered framework that provides scalable, reliable, and high-performance analytics-based data warehouse architecture. In addition to meeting assignment requirements, the design represents contemporary enterprise data engineering standards.

4. Data cleaning & quality management

Quality control and data cleansing is very important to any Big Data Pipeline, and specifically for raw transactions associated with eCommerce. Raw transactions often have a variety of data issues, including missing values, duplicate records, outliers, returned transactions etc. If Data Quality issues are not resolved, the resulting analytics will be inaccurate and lead to bad business decisions.

The Silver Layer within the Bronze -Silver - Gold architecture dedicated to the cleaning, validating and quality reporting for all transactional data from the raw data source. The purpose of Silver was to create an accurate, consistent and analytics ready dataset from the raw transactional data, while also generating Data Quality KPI metrics to track the ongoing improvements in Data Quality.

All of the data cleaning was accomplished via Apache Spark (PySpark) using its distributed processing capabilities. Each of the data cleaning steps utilized a business justification to ensure the accuracy, consistency and analytics readiness of the final dataset.

4.1 Schema Standardization

The first task for this project was to standardize the schemas of each of the raw CSV data sources. Many of these data sources have inconsistent or poorly defined data types which can create problems with analysis. To ensure the data types are consistent and reliable across different datasets, explicit transformations of data types were defined.

- **InvoiceDate** was converted to **TimestampType** using **to_timestamp()** to enable time-based analysis and feature extraction such as month, weekday, and hour.
- **CustomerID** was cast from double to **IntegerType** to maintain a consistent and compact numeric identifier for customers.
- **UnitPrice** was cast to **DecimalType(10,2)** to support precise financial calculations and avoid floating-point rounding errors.

4.2 Handling missing customerID

Missing identification values for customers (CustomerIDs) were highlighted as a major data quality issue as these identifiers are necessary for performing RFM analyses, segmentation and behaviour based analyses, etc. However, if these records were removed, this would also result in considerable loss of data and an inaccurate representation of total revenue.

Instead of dropping these transactions, a surrogate key approach was used. Each invoice that was missing a CustomerID was assigned a unique negative identifier (example -10001, -10002, etc.)

Reasoning

- There are numerous records with missing IDs that contain valid transaction records (product, volume sold, price per unit, date and time of transaction)
- If these records are removed, the revenue trends and the analysis of overall sales performance will be disrupted.
- Using negative surrogate identifiers allows the integrity of revenue to be maintained while clearly separating imputed customers from valid customers.
- By assigning one surrogate identifier to each invoice, it will help prevent merging of random transactions and therefore prevent skewness in RFM metrics.

This approach allows for analytical completeness, protects business measures and ensures that there are clean, distinct relationships between actual customers and imputed customers.

4.3 Handling Negative Quantities (Returns)

The dataset included transactions with negative quantities, representing product returns. Returns affect revenue calculations and require separate treatment to avoid inflating sales figures.

- Negative quantities were flagged as return transactions.
- Returns were separated from standard sales in the Silver layer to enable accurate revenue computation and return analysis.
- These transactions were later used in analytics queries to study return patterns and customer behavior.

This ensures that revenue calculations and product performance metrics reflect only legitimate sales while retaining return data for insight generation.

4.4 Handling Negative Quantities (Returns)

The dataset included transactions with negative quantities, representing product returns. Returns affect revenue calculations and require separate treatment to avoid inflating sales figures.

Approach

- Negative quantities were flagged as return transactions.
- Returns were separated from standard sales in the Silver layer to enable accurate revenue computation and return analysis.
- These transactions were later used in analytics queries to study return patterns and customer behavior.

This ensures that revenue calculations and product performance metrics reflect only legitimate sales while retaining return data for insight generation.

4.5 Identification and Handling of Cancelled Invoices

Invoices with identifiers starting with the letter “C” represent cancelled transactions. Including cancelled invoices in revenue computations would lead to misleading results.

Cleaning Steps

- Cancelled invoices were filtered out from the main sales dataset.
- A separate column IsCancelled was created to flag such records for return and cancellation analytics.
- These records were retained in the dataset for analytical purposes, but excluded from revenue metrics.

This process allows the system to monitor cancellation trends while maintaining accurate sales data.

4.6 Filtering Invalid Prices

Another data quality issue observed was invalid or extreme prices, including negative or zero UnitPrice. Such records are likely data entry errors or test transactions and must be excluded from analytical calculations.

Steps

- Transactions with $\text{UnitPrice} \leq 0$ were filtered out.
- Extreme outliers in UnitPrice or Quantity were identified using logical thresholds and removed.
- These records were documented in the Data Quality Report.

4.7 Duplicate Detection and Removal

Duplicate records were identified as another challenge. Duplicate transactions can occur due to data entry errors or multiple exports from the transactional system.

Methodology

- Duplicate detection was performed based on the combination of InvoiceNo, StockCode, Quantity, Country and InvoiceDate.
- Identified duplicates were removed to avoid skewing revenue and product sales calculations.

Duplicate removal is critical to ensure that aggregated metrics such as total revenue, customer frequency, and product performance are accurate.

4.8 Data Quality Metrics

To monitor and document the effectiveness of the cleaning process, a comprehensive Data Quality Report (DQR) was generated. The DQR included the following metrics

- Total Records
- Records of missing CustomerID
- Return Transactions
- Cancelled Invoices
- Invalid/Negative Prices
- Duplicates Records Removed

This reporting ensures transparency and provides a clear audit trail of the transformations applied to the dataset. It also allows stakeholders to assess the impact of data cleaning on the overall dataset.

5. Feature engineering

Analytics-ready features were created for revenue analysis, customer behavior, product performance, promotions, and engagement metrics that all came from the cleaned Silver layer dataset and were created via feature engineering. The cleaned Silver layer dataset is processed using various PySpark DataFrame operation types, which allow for efficient processing of the cleaned Silver layer dataset using both aggregate and Window Functions. The primary features that were created are outlined below,

5.1 Revenue Calculation

For Each Transaction, Quantity multiplied by UnitPrice computes a Revenue Feature. This Feature is fundamental to all analyses concerning revenue regardless of the level of detail (Transaction, Invoice, Product, or Customer).

5.2 Time-Based Features

Time-based features were extracted from InvoiceDate to support trend and seasonality analysis,

- hour – Hour of the transaction
- weekday – Day of the week (1–7)
- month – Month of the transaction

These features enable monthly, weekly, and hourly revenue analyses, as well as identifying peak sales periods.

5.3 Basket-Level Metrics

Basket-level features were aggregated per invoice to understand purchasing patterns,

- BasketTotalQuantity – Total number of items per invoice
- BasketTotalRevenue – Total revenue per invoice
- UniqueItemsInBasket – Number of distinct products per invoice

These metrics help analyze average basket size, revenue per purchase, and product variety in each transaction.

5.4 Customer RFM Features

Customer-level Recency, Frequency, Monetary (RFM) metrics were created,

- Recency – Days since the last purchase
- Frequency – Number of invoices per customer
- Monetary – Total revenue per customer

RFM features are crucial for segmentation and identifying high-value customers.

5.5 Spark Window-Based Feature

A `customer_rank` feature was added using a Window function to rank customers based on their monetary value. This feature supports targeted marketing and customer prioritization.

5.6 Product-Level Features

Product-level metrics were created to evaluate performance,

- `total_units_sold` – Total quantity sold per product
- `avg_product_price` – Average unit price per product

These metrics are useful for inventory management and revenue analysis by product.

5.7 Customer-Level Features

Additional customer-level features were created for deeper insights,

- `first_purchase_date` – Date of first purchase
- `last_purchase_date` – Date of most recent purchase
- `frequency` – Number of distinct invoices
- `monetary` – Total revenue
- `avg_order_value` – Average revenue per invoice
- `recency_days` – Days since last purchase

5.8 Discount/Promotion Features

Promotion-related features were added to measure high-value transactions,

- `promo_flag` – 1 if transaction revenue > 100, else 0
- `total_high_value_transactions` – Total number of high-value purchases per customer
- `high_value_transaction_pct` – Percentage of transactions considered high value

These features help assess customer responsiveness to promotions and discounts.

5.9 Engagement Features

Customer engagement metrics were created,

- `transactions_per_channel` – Count of distinct invoices per customer

These features provide insights into customer activity levels and repeat purchase behavior.

5.10 Country-Level Features

Aggregated features by country were created for geographical analysis,

- CountryTotalRevenue – Total revenue per country
- CountryTotalQuantity – Total quantity sold per country
- CountryAvgUnitPrice – Average price per country
- CountryCustomerCount – Number of unique customers per country

These metrics help identify top-performing regions and target marketing campaigns geographically.

All features were implemented using PySpark DataFrame operations, aggregations, and Window functions, making the dataset fully analytics-ready for MongoDB storage and further business insights.

6. MongoDB Schema & Indexing

A MongoDB collection designed for fast and easy analytics. It created three collections of features: invoice data factories, customer dim data, and product dim data. The collection contains only invoice-level information with embedded line items. The invoice-level information contains the data necessary to create reports about invoice-level transactions, like the quantity of product purchased, the price per part, the total price of the invoice, and whether or not it has been returned or canceled. The ability to embed line items into the invoice-level structure allows all invoice-level query operations to be performed without multiple joins and facilitates both time and basket-level analysis for revenue aggregation.

Aggregated customer-level metrics are included in the dim_customers collection, such as recency, frequency, monetary value, average order value, promotion-related features, and engagement indicators. This will allow high-value customers to be quickly retrieved, customer segments to be analyzed, churn to be predicted, and targeted marketing analyses to be performed. Similarly, the dim_products collection contains aggregated product-level metrics, including total units sold, average unit price, revenue per country, and return rates. These metrics will allow for insights about the top products sold, product inventory planning, and product performance trends by country.

Indexing was done judiciously across the collections to help speed up query execution time. Key indexes are as follows,

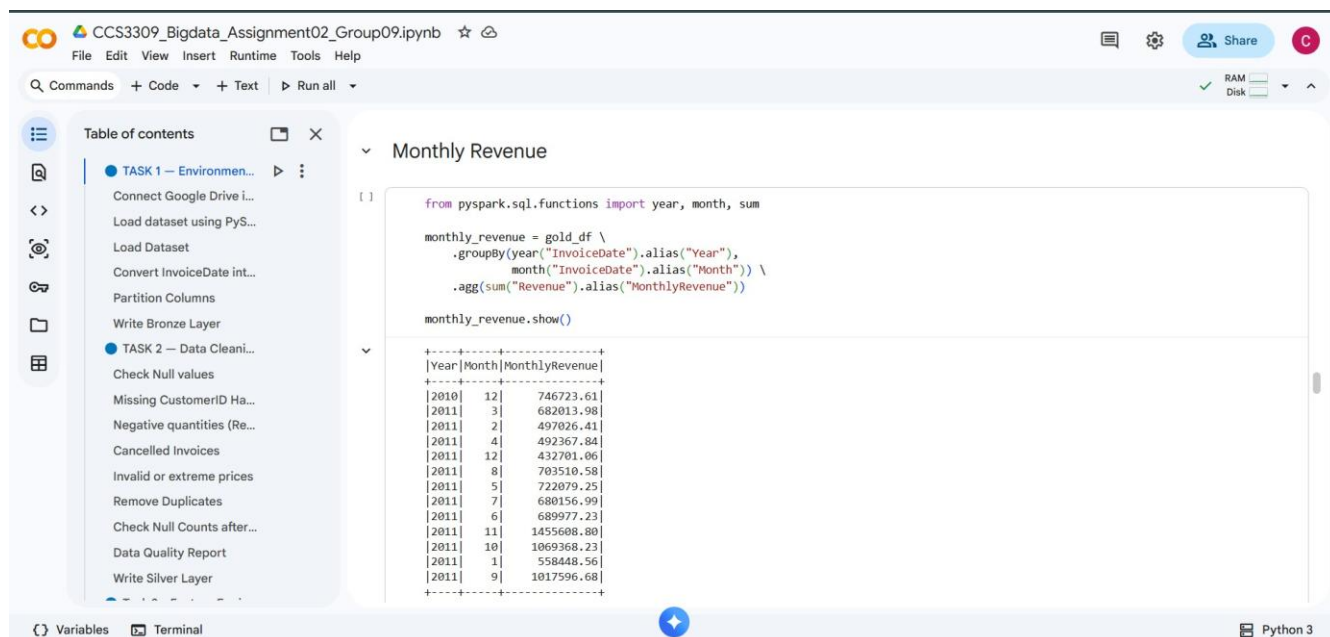
- fact_invoices: Invoice date, CustomerID, Country, LineItems.StockCode
- dim_customers: CustomerID, Recency, Monetary, Frequency
- dim_products: StockCode, TotalUnitsSold, Revenue (Country), and ReturnRate

To speed up ingestion time, bulk writes were used, as well as caching intermediate DataFrames. The combination of schema design and indexing make sure that the MongoDB Gold layer is both scalable, high-performing, and completely capable of providing advanced analytics and business insights.

7. Analytics results

5 Spark-based analytics queries

7.1 Monthly Revenue Trends



The screenshot shows a Jupyter Notebook interface with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The code editor contains a Spark SQL query that groups revenue by year and month, and the output shows a table of monthly revenue data from 2010 to 2011.

```
from pyspark.sql.functions import year, month, sum

monthly_revenue = gold_df \
    .groupBy(year("InvoiceDate").alias("Year"),
            month("InvoiceDate").alias("Month")) \
    .agg(sum("Revenue").alias("MonthlyRevenue"))

monthly_revenue.show()
```

Year	Month	MonthlyRevenue
2010	12	746723.61
2011	3	682013.98
2011	2	497026.41
2011	4	492367.84
2011	12	432701.06
2011	8	703510.58
2011	5	722079.25
2011	7	680156.99
2011	6	689977.23
2011	11	1455608.80
2011	10	1069368.23
2011	1	558448.56
2011	9	1017596.68

Business Insight: Revenue shows strong growth from September to November 2011, peaking in November with over £1.45 million. This significant seasonal spike indicates holiday-driven demand, and the business should prepare with increased inventory and targeted promotions during Q4.

7.2 Top Customers by Spend

Top customers by spend

```
[ ] top_customers = gold_df.groupBy("CustomerID") \
    .agg(sum("Revenue").alias("TotalSpend")) \
    .orderBy(col("TotalSpend").desc())

top_customers.show(10)
```

```
✓ ... +-----+-----+
      |CustomerID|TotalSpend|
      +-----+-----+
      |14646| 279489.02|
      |18102| 256438.49|
      |17450| 187322.17|
      |14911| 132458.73|
      |12415| 123725.45|
      |14156| 113214.59|
      |17511| 88125.38|
      |16684| 65892.08|
      |13694| 62690.54|
      |15311| 59284.19|
      +-----+-----+
      only showing top 10 rows
```

Business Insight: A small group of customers contributes disproportionately to total revenue, with Customer 14646 alone generating over £279K. These high-value customers are prime targets for loyalty programs and personalised retention strategies to protect and grow this critical revenue stream.

7.3 Top Products by Revenue

Top Products

```
[ ] top_products = gold_df.groupBy("StockCode") \
    .agg(sum("Revenue").alias("ProductRevenue")) \
    .orderBy(col("ProductRevenue").desc())

top_products.show(10)
```

```
✓ +-----+-----+
  |StockCode|ProductRevenue|
  +-----+-----+
  |DOT| 206245.48|
  |22423| 164459.49|
  |47566| 98243.88|
  |85123A| 97838.45|
  |85099B| 92175.79|
  |23084| 66661.63|
  |POST| 66230.64|
  |22086| 63715.24|
  |84879| 58792.42|
  |79321| 53746.66|
  +-----+-----+
  only showing top 10 rows
```

Business Insight: A concentrated set of decorative and gift-related products drives the majority of revenue, with StockCode DOT and the REGENCY CAKESTAND 3 TIER leading sales. The business should prioritise consistent stock availability and promotional focus on these top-performing SKUs to maximise revenue.

7.4 Country-Level Sales Analysis

Country Sales

```
country_sales = gold_df.groupBy("Country") \
    .agg(sum("Revenue").alias("CountryRevenue"))
```

```
country_sales.show()
```

```
***
+-----+-----+
| Country | CountryRevenue |
+-----+-----+
| Sweden  | 36585.41 |
| Singapore | 9120.39 |
| Germany | 221509.47 |
| RSA     | 1002.31 |
| France  | 197317.11 |
| Greece  | 4710.52 |
| European Community | 1291.75 |
| Belgium | 40910.96 |
| Finland | 22326.74 |
| Malta   | 2505.47 |
| Unspecified | 4740.94 |
| Italy   | 16890.51 |
| EIRE    | 262993.38 |
| Lithuania | 1661.06 |
| Norway  | 35163.46 |
| Spain   | 54756.03 |
| Denmark | 18768.14 |
| Hong Kong | 9908.24 |
| Iceland | 4310.00 |
| Israel  | 7901.97 |
+-----+-----+
only showing top 20 rows
```

Business Insight: EIRE and Germany are the strongest international markets, generating £262K and £221K respectively, reflecting strong European demand. The business should invest in targeted regional strategies such as localised promotions and expanded distribution in these markets to further grow international revenue.

7.5 Return & Cancellation Patterns

Returns Analysis

```
returns = gold_df.filter(col("Quantity") < 0) \
    .groupBy("Country") \
    .count()
```

```
returns.show()
```

```
***
+-----+-----+
| Country | count |
+-----+-----+
| Sweden  | 11 |
| Germany | 453 |
| France  | 148 |
| Belgium | 38 |
| Finland | 10 |
| Malta   | 15 |
| Italy   | 45 |
| EIRE    | 301 |
| Norway  | 14 |
| Spain   | 48 |
| Denmark | 9 |
| Hong Kong | 4 |
| Israel  | 2 |
| Channel Islands | 10 |
| Cyprus  | 8 |
| Switzerland | 35 |
| Czech Republic | 5 |
| Japan   | 37 |
| Poland  | 11 |
| Portugal | 18 |
+-----+-----+
only showing top 20 rows
```

Business Insight: Germany and EIRE record the highest return counts at 453 and 301 transactions respectively, correlating with their high sales volumes. Investigating the root causes of these returns, whether product quality, delivery issues, or unmet expectations - could significantly reduce revenue loss and improve customer satisfaction in these key markets.

5 MongoDB aggregation pipelines

7.6 Monthly Revenue Trends

Monthly revenue

```
1 list(db.fact_invoices.aggregate([
2   {
3     "$group": {
4       "_id": {"year": "$year", "month": "$month"},
5       "MonthlyRevenue": {"$sum": "$Revenue"}
6     }
7   },
8   {"$sort": {"_id.year": 1, "_id.month": 1}}
9 ]))

... [{"_id": {"year": 2010, "month": 12}, "MonthlyRevenue": 746723.61},
{"_id": {"year": 2011, "month": 1}, "MonthlyRevenue": 558448.5599999999},
{"_id": {"year": 2011, "month": 2}, "MonthlyRevenue": 497026.41},
{"_id": {"year": 2011, "month": 3}, "MonthlyRevenue": 682013.98},
{"_id": {"year": 2011, "month": 4}, "MonthlyRevenue": 492367.84},
{"_id": {"year": 2011, "month": 5}, "MonthlyRevenue": 722079.25},
{"_id": {"year": 2011, "month": 6}, "MonthlyRevenue": 689977.23},
{"_id": {"year": 2011, "month": 7}, "MonthlyRevenue": 680156.99},
{"_id": {"year": 2011, "month": 8}, "MonthlyRevenue": 703510.58},
{"_id": {"year": 2011, "month": 9}, "MonthlyRevenue": 1017596.68},
{"_id": {"year": 2011, "month": 10}, "MonthlyRevenue": 1069368.23},
{"_id": {"year": 2011, "month": 11}, "MonthlyRevenue": 1455608.8},
{"_id": {"year": 2011, "month": 12}, "MonthlyRevenue": 432701.06}]
```

The MongoDB pipeline confirms Spark results exactly, validating both analytical approaches. Revenue peaks in November 2011 at £1.45 million, confirming a strong Q4 seasonal surge that the business should prepare for with advanced inventory and marketing planning. **7.2 Top Customers by Spend**

Top customers

```
1 pipeline = [
2   {
3     "$match": {
4       "InvoiceNo": { "$not": { "$regex": "^C" } }
5     }
6   },
7   {
8     "$group": {
9       "_id": "$CustomerID",
10      "TotalSpend": { "$sum": "$Revenue" }
11    }
12  },
13  { "$sort": { "TotalSpend": -1 } },
14  { "$limit": 10 }
15 ]

result = db.fact_invoices.aggregate(pipeline)

for doc in result:
    print(doc)

... {"_id": 14646, "TotalSpend": 280206.02}
{"_id": 18102, "TotalSpend": 259657.3}
{"_id": 17450, "TotalSpend": 194390.79}
{"_id": 16446, "TotalSpend": 168472.5}
{"_id": 14911, "TotalSpend": 143711.17}
{"_id": 12415, "TotalSpend": 124914.53}
{"_id": 14156, "TotalSpend": 117210.08}
{"_id": 17511, "TotalSpend": 91062.38}
{"_id": 16029, "TotalSpend": 80850.84}
{"_id": 12346, "TotalSpend": 77183.6}
```

By filtering out cancelled invoices, the pipeline accurately reflects true customer value. Customer 14646 leads at £280K, and the top 10 customers represent a highly concentrated revenue source that should be protected through dedicated loyalty and retention programs.

7.3 Top Products by Revenue

Top products

```
j) On
pipeline = [
  {
    "$match": {
      "InvoiceNo": { "$not": { "$regex": "^C" } }
    }
  },
  {
    "$group": {
      "_id": "$StockCode",
      "ProductRevenue": { "$sum": "$Revenue" }
    }
  },
  { "$sort": { "ProductRevenue": -1 } },
  { "$limit": 10 }
]

result = db.fact_invoices.aggregate(pipeline)

for doc in result:
  print(doc)

{'_id': 'DOT', 'ProductRevenue': 206248.77}
{'_id': '22423', 'ProductRevenue': 174156.54}
{'_id': '23843', 'ProductRevenue': 168469.6}
{'_id': '85123A', 'ProductRevenue': 104462.75}
{'_id': '47566', 'ProductRevenue': 99445.23}
{'_id': '85099B', 'ProductRevenue': 94159.81}
{'_id': '23166', 'ProductRevenue': 81700.920000000001}
{'_id': 'POST', 'ProductRevenue': 78101.88}
{'_id': 'M', 'ProductRevenue': 77750.27}
{'_id': '23084', 'ProductRevenue': 66870.03}
```

Excluding cancellations ensures accurate product performance measurement. StockCode DOT and product 22423 dominate revenue, indicating that the business should prioritise these bestselling gift and decorative items in inventory planning and promotional campaigns. **7.4 Country-Level Sales Analysis**

Country sales

```
j) On
pipeline = [
  {
    "$match": {
      "InvoiceNo": { "$not": { "$regex": "^C" } }
    }
  },
  {
    "$group": {
      "_id": "$Country",
      "CountryRevenue": { "$sum": "$Revenue" }
    }
  },
  { "$sort": { "CountryRevenue": -1 } }
]

result = db.fact_invoices.aggregate(pipeline)

for doc in result:
  print(doc)

{'_id': 'United Kingdom', 'CountryRevenue': 9001192.24}
{'_id': 'Netherlands', 'CountryRevenue': 285446.34}
{'_id': 'EIRE', 'CountryRevenue': 283140.52}
{'_id': 'Germany', 'CountryRevenue': 228678.4}
{'_id': 'France', 'CountryRevenue': 209625.37}
{'_id': 'Australia', 'CountryRevenue': 138453.81}
{'_id': 'Spain', 'CountryRevenue': 61558.56}
{'_id': 'Switzerland', 'CountryRevenue': 57067.6}
{'_id': 'Belgium', 'CountryRevenue': 41196.34}
{'_id': 'Sweden', 'CountryRevenue': 38367.03}
{'_id': 'Japan', 'CountryRevenue': 37416.37}
{'_id': 'Norway', 'CountryRevenue': 36165.44}
{'_id': 'Portugal', 'CountryRevenue': 33683.05}
{'_id': 'Finland', 'CountryRevenue': 22546.08}
{'_id': 'Singapore', 'CountryRevenue': 21279.29}
{'_id': 'Channel Islands', 'CountryRevenue': 20440.54}
{'_id': 'Denmark', 'CountryRevenue': 18955.34}
```

The UK dominates at £9 million, confirming the business is primarily UK-centric. However, strong international performance from the Netherlands, EIRE, and Germany highlights significant European market potential worth investing in through targeted regional strategies.

7.5 Return & Cancellation Patterns

Return or cancellation

```
pipeline = [
    {
        "$match": {
            "InvoiceNo": { "$regex": "^C" }, # Only cancelled invoices
            "Quantity": { "$lt": 0 }        # Returned items
        }
    },
    {
        "$group": {
            "_id": "$Country",
            "ReturnCount": { "$sum": 1 }
        }
    },
    { "$sort": { "ReturnCount": -1 } }
]

result = db.fact_invoices.aggregate(pipeline)

for doc in result:
    print(doc)
```

```
{'_id': 'United Kingdom', 'ReturnCount': 7821}
{'_id': 'Germany', 'ReturnCount': 453}
{'_id': 'EIRE', 'ReturnCount': 301}
{'_id': 'France', 'ReturnCount': 148}
{'_id': 'USA', 'ReturnCount': 112}
{'_id': 'Australia', 'ReturnCount': 74}
{'_id': 'Spain', 'ReturnCount': 48}
{'_id': 'Italy', 'ReturnCount': 45}
{'_id': 'Belgium', 'ReturnCount': 38}
{'_id': 'Japan', 'ReturnCount': 37}
{'_id': 'Switzerland', 'ReturnCount': 35}
```

The UK leads returns at 7,821 transactions, proportional to its sales volume, while Germany and EIRE follow with 453 and 301 respectively. Investigating return root causes and implementing structured returns management could meaningfully recover revenue and reduce operational costs.

8. Performance optimization

The importance of performance optimization was a significant concern for this project, as the ability to process large amounts of transactional data using PySpark efficiently is essential considered critical in this project. Since the ETL pipeline performed revenue calculations, as well as aggregations, window

functions, and prepared data to be inserted into MongoDB, it was necessary to optimize Spark's operations so that the execution time and resource consumption would be minimized. Some of the key optimization techniques that were used included partitioning strategies, caching and persistence, and the use of broadcast joins.

There are several uses of partitioning strategy for the ETL pipeline to assist with parallel processing and minimize data shuffling across the cluster. Because Spark distributes data amongst partitions, if managed correctly, partitioning can improve hardware utilization by providing balanced distribution of the workload as well as decreasing the overall execution time. For example, when performing aggregations and feature engineering on the data, the data in each partition was repartitioned by frequent grouping keys, such as CustomerID, InvoiceNo., and Country. As a result, the performance was increased for the customer-level, invoice-level, and country-level computations performed on the data.

Moreover, the way that the processed data needed to be written to the storage or prepared for insertion into MongoDB also required associated adjustments within the partitions to help prevent the small file issue and uneven distribution of the tasks distributed across the cluster. Proper partitioning allowed for a reduction in shuffle operations, less overall hindrance to execution processes, and improved scalability of the overall ETL pipeline.

Caching and Persistence

The cached intermediate DataFrames are reused throughout analytics computations/feature-testing. For example, the revenue calculated DataFrame is frequently reused for basket metrics, RFM analysis, product feature calculations, and country aggregations. Rather than perform the transformation again and again, caching stores the intermediate result in memory and saves execution time.

Persisting some DataFrame's data was done selectively to balance memory consumption and computation. DataFrames with many dependent downstream processes were marked for caching, eliminating excess memory consumption. This provided greater efficiency while maintaining system reliability; therefore, the intermediate transformation process was facilitated.

Broadcast Joins

Broadcast joins were applied in optimization of join(s) when one dataset was substantially smaller than the other. In the event of a large join, join(s) will cause substantial data shuffling, thus increasing execution time. Spark broadcasts smaller DataFrames (aggregate customer metrics/customer aggregations or aggregate product metrics) to all worker nodes, thus avoiding the need for shuffle-heavy joins when executing the smaller DataFrames.

Broadcast-join strategy was used in executing join(s) between feature-engineered DataFrames and smaller aggregations of data. The implementation of this broadcast-join strategy reduced the amount of network processing required, decreased the number of shuffle stages, and enhanced overall query execution performance.

9. Challenges & lessons learned

While creating the BSG Data Pipeline many difficulties arose both due to technical issues and human factors. One of the biggest challenges was how to deal with missing and inconsistent data in the raw eCommerce dataset. This was caused by the fact that the raw eCommerce dataset contained null customer IDs, cancelled transactions, returns, negative quantities and unrealistically high unit prices. Therefore, several times prior to processing the Data, systematic methods had to be developed to clean, validate and de-duplicate the Data which required thoughtful planning to ensure only high quality Data would be forwarded for further processing.

Creating features and computing them in large volumes of transactional data proved to be very technically challenging. Aggregating metrics at several levels, including revenues, basket details, RFM scores for customers, product performance, and country-level analytics, required effective application of PySpark functions. The application of Window operations was also essential for efficient computation. Scaling these computed results was critical due to the diminished execution time requirements and memory issues that we encountered during testing, thus emphasizing the need for scalable, distributed processing methods.

The development of the MongoDB Gold Layer Schema and Indexing Strategy was a careful and laborious effort that included balancing whether to embed or reference data; determining which fields needed to have indexes applied to them; and testing the performance of aggregation queries. Each task was performed iteratively through experimentation with the steps taken showing mistakes were made when mislabeling columns, overwriting intermediate results, referencing incorrect DataFrames, etc., which represent typical human errors in an academic student-led project. Debugging these problems highlighted the importance of doing version control, performing systematic testing, and having clear documentation.

Important lessons were learned as a result of the combined technical and human challenges experienced on this project,

- **Quality Data is One of the Foundations for Analytics:** Good Analytics requires that data be cleaned and validated prior to analysis and that nulls and outliers be handled.
- **Feature Engineering Must Be Scalable:** Using distributed and/or Window functions will facilitate the efficient processing of large datasets.
- **Schema Design Must Support the Queries That Will Be Executed Against the Data:** Good collection and indexing design will improve the performance of analytical queries.
- **Performance Optimization Is Critical:** Use partitioning, caching, and bulk operations to ensure that you can work with large-scale datasets.
- **Having A High Level of Attention To Detail Is Important:** Make sure you debug carefully and keep track of different versions of code to avoid unintended propagation of errors.
- **Document, Document, Document!** Keeping good notes and following a consistent workflow will help support good teamwork and maintainability.

In summary, this project illustrates how both technical and human factors can affect the outcome of a real-world data engineering project. Overcoming these challenges has reinforced best practices for being able to design scalable ETL processes, perform reliable analytics, and manage the workflows in a disciplined manner.

10. Conclusion & future work

The delivery of a complete Bronze/Silver/Gold data pipeline encompassing the ingestion of raw e-commerce transaction data, processing of that data (cleaning and quality control), and the creation of analytics-ready collections in MongoDB, that produce output that is both timely and accurate. This was achieved using an ETL process to transform raw transactional data into useful business intelligence.

The structure of the dataset provides a framework for scaling and providing high temperature performance analytic functions to support revenue calculations, time-based features, transaction attributes, customer RFM analysis, product performance metrics, promotional metrics, and customer engagement features. The design of the MongoDB schema and indexing strategy provides an efficient method for querying the dataset and Spark provides the infrastructure necessary for distributed processing of very large transaction datasets. The result is a demonstration of how raw transaction datasets can be converted to meaningful business intelligence through structured ETL processes and a focus on performance-aware designs.

The results of the analysis provided valuable insight into revenue trends, customer segments, product performance, sales distribution by country, and return data patterns. Additionally, high value customers and top-performing products were identified, as well as identifying seasonal revenue peaks and measuring the effectiveness of promotions. These findings highlight the need for business entities to integrate one or more data engineering methods with analytical modeling techniques to inform strategic business decision-making.

We could make several improvements and enhancements to the system from now going forward.

- We can improve the system by introducing real-time data ingestion by integrating additional streaming tools including, for example, Kafka, which will allow us to process transactions in real-time.
- Implementing advanced analytics and machine learning techniques including churn prediction models, recommendation systems, and demand forecasting models.
- Integrating our dashboarding tools (i.e., connecting MongoDB to an external visualization tool such as Power BI or Tableau) into the overall architecture of the pipeline will provide users with the ability to view and produce interactive reports.
- We could incorporate workflow automation/orchestration tools (including tools like Airflow) to automate our ETL scheduling process.

- Additionally, moving forward we could deploy the entire pipeline to cloud-based platforms. Doing this would enable us to take advantage of cloud computing's improved scalability, availability, reliability, etc.

Finally, we could enhance our data governance capabilities by providing additional data validation rules, monitoring dashboards, and logging frameworks in order to achieve the high-level reliability expected by production solutions.

In summary, this project has demonstrated the practical implementation of a large-scale data engineer pipeline capable of integrating distributed processing along with NoSQL-based analytic data storage. Beyond demonstrating the technical implementation aspects of a scalable solution, the project also emphasizes additional considerations which must be accounted for in order to transform raw data into actionable intelligence, including the importance of performance tuning for optimal SQL execution, good schema design, and disciplined workflow management practices.

11. References

Kaggle Dataset - UK-based online retail transactional dataset (2010–2011)
<https://www.kaggle.com/datasets/roikb55/online-retail-xlsx>

Apache Spark Documentation <https://spark.apache.org/docs/>

MongoDB Atlas Documentation
<https://www.mongodb.com/cloud/atlas>

PySpark API Reference
<https://spark.apache.org/docs/latest/api/python/index.html>

Building End-to-End Data Pipelines: From Data Ingestion to Analysis
<https://www.kdnuggets.com/building-end-to-end-data-pipelines-from-data-ingestion-to-analysis>
Overview of Data Pipeline
<https://www.geeksforgeeks.org/software-engineering/overview-of-data-pipeline/>